

Tutorial 3 – Create a Jigsaw Board with Image

Member Variable

Let's add a variable `public String imageFilename` as this variable will hold the filename of the image that will be used to generate the Jigsaw board.

Load Base Texture

Now, we will create a function called `LoadBaseTexture` as this function will load the generating a `Tile` object at the specified position `i, j` in the jigsaw puzzle grid, based on the provided `baseTexture`.

Next, we w

Start Method

Create Transparent View

Set Camera Position

Create Jigsaw Tile

Create Game Object From Tile

Create Jigsaw Tiles

Next, we will create the CreateJigsawTiles function as it generates the jigsaw puzzle tiles based on the opaque base texture.

```
void CreateJigsawTiles()
{
    Texture2D baseTexture = mBaseSpriteOpaque.texture;
    numTileX = baseTexture.width / Tile.tileSize;
    numTileY = baseTexture.height / Tile.tileSize;

    mTiles = new Tile[numTileX, numTileY];
    mTileGameObjects = new GameObject[numTileX, numTileY];

    for (int i = 0; i < numTileX; i++)
    {
        for (int j = 0; j < numTileY; j++)
        {
            mTiles[i, j] = CreateTile(i, j, baseTexture);
            mTileGameObjects[i, j] = CreateGameObjectFromTile(mTiles[i, j]);
            if (parentForTiles != null)
            {
                mTileGameObjects[i, j].transform.SetParent(parentForTiles);
            }
        }
    }
}
```

Then, we calculate the number of tiles in both the horizontal and vertical axes by dividing the width and height of the base texture by the tileSize.

Next, we will calculate the array mTiles and mTileGameObjects to store Tile objects and corresponding Game Objects instances for each tile position.

Afterwards, we create a game object using the CreateGameObjectFromTile function.

Create Tile

The CreateTile method generates a Tile object at the specified position l, j in the jigsaw puzzle grid, based on the provided baseTexture.

```

Tile CreateTile(int i, int j, Texture2D baseTexture)
{
    Tile tile = new Tile(baseTexture);
    tile.xIndex = i;
    tile.yIndex = j;

    // Left side tiles.
    if (i == 0)
    {
        tile.SetCurveType(Tile.Direction.LEFT, Tile.PosNegType.NONE);
    }
    else
    {
        // We have to create a tile that has LEFT direction opposite curve type.
        Tile leftTile = mTiles[i - 1, j];
        Tile.PosNegType rightOp = leftTile.GetCurveType(Tile.Direction.RIGHT);
        tile.SetCurveType(Tile.Direction.LEFT, rightOp == Tile.PosNegType.NEG ?
            Tile.PosNegType.POS : Tile.PosNegType.NEG);
    }

    // Bottom side tiles
    if (j == 0)
    {
        tile.SetCurveType(Tile.Direction.DOWN, Tile.PosNegType.NONE);
    }
    else
    {
        Tile downTile = mTiles[i, j - 1];
        Tile.PosNegType upOp = downTile.GetCurveType(Tile.Direction.UP);
        tile.SetCurveType(Tile.Direction.DOWN, upOp == Tile.PosNegType.NEG ?
            Tile.PosNegType.POS : Tile.PosNegType.NEG);
    }

    // Right side tiles.
    if (i == numTileX - 1)
    {
        tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.NONE);
    }
    else
    {
        float toss = Random.Range(0f, 1f);
        if (toss < 0.5f)
        {
            tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.POS);
        }
        else
        {
            tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.NEG);
        }
    }
}

```

```

// Up side tile.
if (j == numTileY - 1)
{
    tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.NONE);
}
else
{
    float toss = Random.Range(0f, 1f);
    if (toss < 0.5f)
    {
        tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.POS);
    }
    else
    {
        tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.NEG);
    }
}

tile.Apply();
return tile;

```

Now,

Otherwise,

Finally, we apply the changes made to the tile's curve types using the Apply method as it'll finalize the finalCut image with flood fill.

Now, we will go to the Start method and add the function call to CreateJigsawTiles.

```
// Start is called before the first frame update
@ Unity Message | 0 references
void Start()
{
    mBaseSpriteOpaque = LoadBaseTexture();
    mGameObjectOpaque = new GameObject();
    mGameObjectOpaque.name = imageFilename + "_Opaque";
    mGameObjectOpaque.AddComponent<SpriteRenderer>().sprite = mBaseSpriteOpaque;
    mGameObjectOpaque.GetComponent<SpriteRenderer>().sortingLayerName = "Opaque";

    mBaseSpriteTransparent = CreateTransparentView(mBaseSpriteOpaque.texture);
    mGameObjectTransparent = new GameObject();
    mGameObjectTransparent.name = imageFilename + "_Transparent";
    mGameObjectTransparent.AddComponent<SpriteRenderer>().sprite = mBaseSpriteTransparent;
    mGameObjectTransparent.GetComponent<SpriteRenderer>().sortingLayerName = "Transparent";

    mGameObjectOpaque.gameObject.SetActive(false);

    SetCameraPosition();

    // Create the Jigsaw tiles.
    //CreateJigsawTiles();
    StartCoroutine(Coroutine_CreateJigsawTiles());
}
```

Go to the Unity editor click play then you'll be able to see all the jigsaw pieces created from the image

Enchantments

Now, we'll show the animated effect of creating the jigsaw tiles. As we will copy and paste the CreateJigsawTiles method and convert it to a coroutine. The name will be changed to Coroutine_CreateJigsawTiles with a return type of IEnumerator.

Afterwards, go to the start method replace CreateJigsaw call and replace it with StartCoroutine with the input parameter as Coroutine_CreateJigsawTiles.

Go to the Unity editor click play then you'll be able to see animated effect of all the jigsaw pieces to show an image.

Final Script:

Board

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

© Unity Script | 0 references
public class BoardGen : MonoBehaviour
{
    public string imageFilename;
    Sprite mBaseSpriteOpaque;
    Sprite mBaseSpriteTransparent;

    GameObject mGameObjectOpaque;
    GameObject mGameObjectTransparent;

    public float ghostTransparency = 0.1f;

    // Jigsaw tiles creation.
    9 references
    public int numTileX { get; private set; }
    9 references
    public int numTileY { get; private set; }

    Tile[,] mTiles = null;
    GameObject[,] mTileGameObjects = null;

    public Transform parentForTiles = null;

    1 reference
    Sprite LoadBaseTexture()
    {
        Texture2D tex = SpriteUtils.LoadTexture(imageFilename);
        if (!tex.isReadable)
        {
            Debug.Log("Error: Texture is not readable");
            return null;
        }

        if (tex.width % Tile.tileSize != 0 || tex.height % Tile.tileSize != 0)
        {
            Debug.Log("Error: Image must be of size that is multiple of <" + Tile.tileSize + ">");
            return null;
        }

        // Add padding to the image.
        Texture2D newTex = new Texture2D(
            tex.width + Tile.padding * 2,
            tex.height + Tile.padding * 2,
            TextureFormat.ARGB32,
            false);
    }

```

```

    // Set the default colour as white
    for (int x = 0; x < newTex.width; ++x)
    {
        for (int y = 0; y < newTex.height; ++y)
        {
            newTex.SetPixel(x, y, Color.white);
        }
    }

    // Copy the colours.
    for (int x = 0; x < tex.width; ++x)
    {
        for (int y = 0; y < tex.height; ++y)
        {
            Color color = tex.GetPixel(x, y);
            color.a = 1.0f;
            newTex.SetPixel(x + Tile.padding, y + Tile.padding, color);
        }
    }
    newTex.Apply();

    Sprite sprite = SpriteUtils.CreateSpriteFromTexture2D(
        newTex,
        0,
        0,
        newTex.width,
        newTex.height);
    return sprite;

```

```

// Start is called before the first frame update
@ Unity Message | 0 references
void Start()
{
    mBaseSpriteOpaque = LoadBaseTexture();
    mGameObjectOpaque = new GameObject();
    mGameObjectOpaque.name = imageFilename + "_Opaque";
    mGameObjectOpaque.AddComponent<SpriteRenderer>().sprite = mBaseSpriteOpaque;
    mGameObjectOpaque.GetComponent<SpriteRenderer>().sortingLayerName = "Opaque";

    mBaseSpriteTransparent = CreateTransparentView(mBaseSpriteOpaque.texture);
    mGameObjectTransparent = new GameObject();
    mGameObjectTransparent.name = imageFilename + "_Transparent";
    mGameObjectTransparent.AddComponent<SpriteRenderer>().sprite = mBaseSpriteTransparent;
    mGameObjectTransparent.GetComponent<SpriteRenderer>().sortingLayerName = "Transparent";

    mGameObjectOpaque.gameObject.SetActive(false);

    SetCameraPosition();

    // Create the Jigsaw tiles.
    //CreateJigsawTiles();
    StartCoroutine(Coroutine_CreateJigsawTiles());
}

```

```

Sprite CreateTransparentView(Texture2D tex)
{
    Texture2D newTex = new Texture2D(
        tex.width,
        tex.height,
        TextureFormat.ARGB32,
        false);

    for (int x = 0; x < newTex.width; x++)
    {
        for (int y = 0; y < newTex.height; y++)
        {
            Color c = tex.GetPixel(x, y);
            if (x > Tile.padding &&
                x < (newTex.width - Tile.padding) &&
                y > Tile.padding &&
                y < (newTex.height - Tile.padding))
            {
                c.a = ghostTransparency;
            }
            newTex.SetPixel(x, y, c);
        }
    }
    newTex.Apply();

    Sprite sprite = SpriteUtils.CreateSpriteFromTexture2D(
        newTex,
        0,
        0,
        newTex.width,
        newTex.height);
    return sprite;
}

```



```

void SetCameraPosition()
{
    Camera.main.transform.position = new Vector3(mBaseSpriteOpaque.texture.width / 2,
        mBaseSpriteOpaque.texture.height / 2, -10.0f);
    Camera.main.orthographicSize = mBaseSpriteOpaque.texture.width / 2;
}

2 references
public static GameObject CreateGameObjectFromTile(Tile tile)
{
    GameObject obj = new GameObject();

    obj.name = "TileGameObe_" + tile.xIndex.ToString() + "_" + tile.yIndex.ToString();

    obj.transform.position = new Vector3(tile.xIndex * Tile.tileSize, tile.yIndex * Tile.tileSize, 0.0f);

    SpriteRenderer spriteRenderer = obj.AddComponent<SpriteRenderer>();
    spriteRenderer.sprite = SpriteUtils.CreateSpriteFromTexture2D(
        tile.finalCut,
        0,
        0,
        Tile.padding * 2 + Tile.tileSize,
        Tile.padding * 2 + Tile.tileSize);

    BoxCollider2D box = obj.AddComponent<BoxCollider2D>();
    return obj;
}

0 references
void CreateJigsawTiles()
{
    Texture2D baseTexture = mBaseSpriteOpaque.texture;
    numTileX = baseTexture.width / Tile.tileSize;
    numTileY = baseTexture.height / Tile.tileSize;

    mTiles = new Tile[numTileX, numTileY];
    mTileGameObjects = new GameObject[numTileX, numTileY];

    for (int i = 0; i < numTileX; i++)
    {
        for (int j = 0; j < numTileY; j++)
        {
            mTiles[i, j] = CreateTile(i, j, baseTexture);
            mTileGameObjects[i, j] = CreateGameObjectFromTile(mTiles[i, j]);
            if (parentForTiles != null)
            {
                mTileGameObjects[i, j].transform.SetParent(parentForTiles);
            }
        }
    }
}

```

```

Tile CreateTile(int i, int j, Texture2D baseTexture)
{
    Tile tile = new Tile(baseTexture);
    tile.xIndex = i;
    tile.yIndex = j;

    // Left side tiles.
    if (i == 0)
    {
        tile.SetCurveType(Tile.Direction.LEFT, Tile.PosNegType.NONE);
    }
    else
    {
        // We have to create a tile that has LEFT direction opposite curve type.
        Tile leftTile = mTiles[i - 1, j];
        Tile.PosNegType rightOp = leftTile.GetCurveType(Tile.Direction.RIGHT);
        tile.SetCurveType(Tile.Direction.LEFT, rightOp == Tile.PosNegType.NEG ?
            Tile.PosNegType.POS : Tile.PosNegType.NEG);
    }

    // Bottom side tiles
    if (j == 0)
    {
        tile.SetCurveType(Tile.Direction.DOWN, Tile.PosNegType.NONE);
    }
    else
    {
        Tile downTile = mTiles[i, j - 1];
        Tile.PosNegType upOp = downTile.GetCurveType(Tile.Direction.UP);
        tile.SetCurveType(Tile.Direction.DOWN, upOp == Tile.PosNegType.NEG ?
            Tile.PosNegType.POS : Tile.PosNegType.NEG);
    }

    // Right side tiles.
    if (i == numTileX - 1)
    {
        tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.NONE);
    }
    else
    {
        float toss = Random.Range(0f, 1f);
        if (toss < 0.5f)
        {
            tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.POS);
        }
        else
        {
            tile.SetCurveType(Tile.Direction.RIGHT, Tile.PosNegType.NEG);
        }
    }
}

```

```

// Up side tile.
if (j == numTileY - 1)
{
    tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.NONE);
}
else
{
    float toss = Random.Range(0f, 1f);
    if (toss < 0.5f)
    {
        tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.POS);
    }
    else
    {
        tile.SetCurveType(Tile.Direction.UP, Tile.PosNegType.NEG);
    }
}

tile.Apply();
return tile;

```